

CMTAT ⇄ Stablecoin Feature Comparison

Version 0.1.0

Feature-by-feature comparison between **CMTAT** (the CMTA Token standard, Solidity/EVM implementation) and seven production stablecoins whose source is vendored in this repository.

The central question this document answers: **for each feature a real stablecoin ships, does CMTAT have it, and if not in the token itself, does one of the seven CMTA companion projects provide it?**

Contents

- [Sources and versions](#)
 - [1.1 CMTAT and its companion projects](#)
 - [1.2 Third-party dependencies read directly](#)
 - [1.3 Stablecoins analysed](#)
- [How to read the tables](#)
- [The CMTAT stack](#)
- [Comparison — token standards & signature flows](#)
- [Comparison — supply control](#)
- [Comparison — compliance & enforcement](#)
- [Comparison — access control & governance](#)
- [Comparison — upgradeability & lifecycle](#)
- [Comparison — yield, cross-chain and extras](#)
- [Gap analysis](#)

This document compares **CMTAT against the stablecoins**. For a stablecoin-to-stablecoin comparison that does not centre on CMTAT, see [vendor-stablecoins/SUMMARY.md](#).

1. Sources and versions

Every claim below was read from the code pinned in this repository, not from marketing material or third-party summaries. All submodule pins are the commits recorded in this repository's index, taken from `git submodule status`; run `git submodule update --init --recursive` to reproduce them.

1.1 CMTAT and its companion projects

Everything under [cmtat/](#): the token standard, the seven companion projects that extend it, three non-Solidity implementations (`CMTAT-Confidential`, `private-CMTAT-aztec`, `CMTAT-Tezos-FA2`), and `CMTAT-Solana`, which is a written specification rather than code.

Project	Role	Tag	Commit	Date	Solidity
CMTAT	the token standard	<code>v3.3.0-rc3</code>	<code>658672f</code>	2026-07-31	0.8.36

Project	Role	Tag	Commit	Date	Solidity
RuleEngine	transfer-restriction controller	v3.0.0-rc6	ca75429	2026-08-21	^0.8.20 (compiled 0.8.36)
Rules	the 15 pluggable rules	v0.6.0	283efe7	2026-08-24	0.8.36
SnapshotEngine	on-chain balance snapshots	v0.5.0	aa08935	2026-06-09	0.8.34
CMTAT-Factory	CREATE2 proxy factories	v0.5.0	46ca1ec	2026-08-26	0.8.36
CMTAT-LayerZero	LayerZero V2 OFT adapters	v0.2.0 +1	e57ca4f	2026-02-16	0.8.33
CMTAT-ACE	Chainlink ACE policy-engine token builds	v0.3.0	34fcb41	2026-06-26	0.8.x
CMTAT-CCIP	Chainlink CCIP deployment scripts	(untagged)	c4f946d	2025-12-01	0.8.x
CMTAT-Confidential	Zama FHE confidential variant	v1.0.0	285ed93	2026-07-10	0.8.x
private-CMTAT-aztec	private CMTAT on the Aztec L2 — prototype	(untagged)	61f4220	2025-07-28	— (Noir)
CMTAT-Tezos-FA2	CMTAT on Tezos, built on the FA2 standard	v1.0 +1	8953de2	2023-02-23	— (SmartPy)
CMTAT-Solana	specification for a CMTAT-compliant token on Solana	v1.0.0	f8fdad8	2025-12-08	— (no code)

The CMTAT and RuleEngine pins are **release candidates**: their `version()` strings report `3.3.0` and `3.0.0`, but the tagged commits are `v3.3.0-rc3` and `v3.0.0-rc6`.

Neither Chainlink integration is audited. `CMTAT-ACE` states it has had **no formal audit**, static analysis and AI review only; `CMTAT-CCIP` is unaudited testnet tooling derived from Chainlink's examples repository.

CMTAT in production as a stablecoin. Zand Trust (2025) issued an AED stablecoin using CMTAT v3.0.0 via Taurus infrastructure ([Zand Trust](#)).

1.2 Third-party dependencies read directly

Project	Why it is here	Tag	Commit	Date
chainlink-ace	The ACE <code>PolicyEngine</code> and its policy library, which <code>CMTAT-ACE</code> attaches but does not contain	v1.2.0 +1	60f0450	2026-07-23

Every ACE policy named in this document (`VolumeRatePolicy`, `SecureMintPolicy`, `PausePolicy`, `MaxPolicy`, `VolumePolicy`, `IntervalPolicy`, `RoleBasedAccessControlPolicy`, `AllowPolicy`, `RejectPolicy`, `BypassPolicy`) was read from `packages/policy-management/src/policies/`.

1.3 Stablecoins analysed

Everything under [vendor-stablecoins/](#): four upstream repositories pinned as submodules and three verified-source dumps taken from Etherscan. The EURR dump is the only one that captures two contracts: the token and the `AuthRegistry` it delegates compliance to.

Issuer	Token(s)	Kind	Tag	Commit / address	Date	Solidity
Circle	USDC, EURC	submodule	<code>release-2026-08-12T202509</code>	<code>fc85788</code>	2026-08-12	0.6.12
Paxos	USDP, USDG, PYUSD, PAXG	submodule	<code>v2.1.0+3</code>	<code>674ac10</code>	2026-07-29	0.8.28
Monerium	EURc, GBPc, USDc, ISKc	submodule	<code>v2.0.0+24</code>	<code>514bee7</code>	2025-08-21	0.8.x
Wyoming	FRNT, wFRNT	submodule	<i>(untagged)</i>	<code>f8aa140</code>	2026-04-30	0.8.22
SG-FORGE	CoinVertible EURCV, USDCV	Etherscan dump	—	impl. <code>0xF4ccC80C...</code>	dumped 2026-08-26	0.8.22
Bridge (Stripe)	EURR — Revolut Euro	Etherscan dump	—	token impl. <code>0xb6De5eAb...</code> + registry <code>0x73531fc8...</code>	dumped 2026-08-26	^0.8.24
Tether	USDT	Etherscan dump	—	<code>0xdac17f95...</code>	dumped 2026-08-26	0.4.17

The Etherscan dumps capture a single **implementation** contract each, not a full repository: no tests, scripts or history. See [vendor-stablecoins/README.md](#).

This is a snapshot, not the live on-chain state. Every statement about a stablecoin below describes the source pinned in the table above, at the date shown, and may already differ from what is deployed.

- **Six of the seven sit behind upgradeable proxies** — transparent for USDC, UUPS for Monerium, Wyoming, CoinVertible and EURR, and both across the Paxos tokens. The implementation an address serves today can be replaced with no change to the repository this analysis reads.
- **USDT is not upgradeable**, but `deprecate()` can redirect every entry point to a successor contract, which is a larger change than an implementation swap.
- **The upstream repositories move on**, and each of the three Etherscan dumps was taken on a single day, from whichever implementation the proxy pointed at then.

Re-check the deployed bytecode before relying on any of this operationally.

Wyoming — `frontier-stable-token`. The Commission announced a migration from LayerZero to **Chainlink CCIP** in August 2026 ([press release](#)). This snapshot still reflects the LayerZero architecture, so every FRNT / wFRNT cross-chain statement below describes the OFT design, not the one now in production.

2. How to read the tables

CMTAT is not one contract, so each table splits it into **three** columns. Light is kept separate because it is a deliberately reduced build, not a subset of the others: separating it shows at a glance which features an issuer gives up by choosing the minimal contract, and which need a companion project on top of a heavier one.

Column	Meaning
Light	CMTATStandaloneLight / CMTATUpgradeableLight — a deliberately minimal CMTAT for issuers who want fewer features and a smaller contract, and the variant CMTA recommends for stablecoins. 11.3 KiB deployed, roughly half the Standard variant, per CMTAT's own documentation (sizes were not recompiled here).
CMTAT	The token contract in any variant other than Light: <code>Standard</code> , <code>Permit</code> , or a dedicated one (<code>Allowlist</code> , <code>Snapshot</code> , <code>ERC1363</code> , <code>ERC7551</code> , <code>HolderList</code> , <code>Debt</code> , <code>UUPS</code>). The applicable variant is named in the cell.
Companion	A separate CMTA project that extends the token: RuleEngine , Rules , SnapshotEngine , CMTAT-Factory , CMTAT-LayerZero , CMTAT-ACE or CMTAT-CCIP . The specific contract or policy is named.

What minimalism costs: Light cannot reach the companion contracts

Keeping Light small means leaving out the setters the companion projects plug into, so most of the Companion column is unavailable to it. **CMTAT-Factory is the only companion Light can definitely use.** (Evidence, for readers who want it: Light is built on [0_CMTATBaseCore](#), every other variant on a larger base.)

Companion	From Light?	Why
RuleEngine + all 15 <code>Rules</code>	✗	Light has no <code>setRuleEngine</code> ; binding a rule straight to the token needs that same setter
SnapshotEngine	✗	Light has no <code>setSnapshotEngine</code> , and the engine's in-token variants are built on a heavier CMTAT base
CMTAT-ACE	✗	it is a separate token build, and starts from a heavier CMTAT base
CMTAT-CCIP	✗	needs the ERC-7802 cross-chain entry points, which Light does not implement
CMTAT-LayerZero	⚠	its recommended adapter needs ERC-7802, which Light lacks; the fallback adapter needs only <code>mint</code> / <code>burn</code> , which Light has, but that pairing is untested

Companion	From Light?	Why
CMTAT-Factory	✓ yes	<code>CMTAT_LIGHT_TP_FACTORY</code> , <code>CMTAT_LIGHT_BEACON_FACTORY</code> (no Light UUPS factory)

Read the Companion column as: "available to CMTAT, but only if you move off Light."

Symbols: ✓ available · ⚠ partial, indirect or non-standard · ✗ absent · ✓ companion feature reachable from Light (CMTAT-Factory only).

Stablecoin column abbreviations: **USDC** = Circle, **PAX** = Paxos, **MON** = Monerium, **FRNT** = Wyoming, **CV** = CoinVertible, **EURR** = Revolut Euro, **USDT** = Tether.

3. The CMTAT stack

CMTAT is deliberately split in two directions, and knowing what each part owns is the key to reading the tables.

- **Across repositories** — the token plus the seven companion projects that plug into it. The table below is the inventory.
- **Within the token** — 25 modules under `contracts/modules/wrapper/` (core, extensions, options, controllers, security), sitting behind a layered set of base contracts.
 - Each of the 21 shipped deployment variants is a thin assembly of those parts. `CMTATStandardStandalone` is 66 lines: inherit a base, add a rule-engine hook and a forwarder.
 - An issuer whose requirements match no shipped variant composes their own from the same parts rather than forking the token, and the same seam lets an integrator graft on features CMTAT does not have.
 - Two companion projects are exactly that: `SnapshotEngine`'s in-token variants and every `CMTAT-ACE` build are third-party assemblies of CMTAT base contracts.
- **None of the seven stablecoins is factored this way.** Each is a fixed inheritance chain, and adding a feature means editing the token.

Project	What it owns	Bound to the token by	Light?
CMTAT	ERC-20, mint/burn, pause, address freeze, forced burn/transfer, RBAC, deactivation, documents, cross-chain entry points	— (it is the token)	—
RuleEngine	The controller: holds an ordered list of rules, is called on every transfer/mint/burn, returns the first non-zero ERC-1404 code or reverts	<code>token.setRuleEngine(engine)</code> <code>(DEFAULT_ADMIN_ROLE)</code>	✗ no setter
Rules	The 15 pluggable rules: whitelists, blacklist, sanctions oracle, supply caps, balance caps, proof of reserve, mint allowance, conditional transfer, identity registry	<code>engine.setRules([...])</code> , or a single rule bound directly with <code>setRuleEngine(rule)</code>	✗ needs the setter
SnapshotEngine	Scheduled on-chain balance snapshots for dividend/reward distribution; external engine or compiled into the token	<code>token.setSnapshotEngine(engine)</code> , or the <code>CMTAT*InternalSnapshot</code> variants	✗ neither path exists in Light
CMTAT-Factory	Deterministic (CREATE2) deployment behind Transparent, UUPS or Beacon proxies	deployment-time	✓ <code>CMTAT_LIGHT_*_FACTORY</code>
CMTAT-LayerZero	LayerZero V2 OFT adapters (mint/burn, not lock/unlock), each with its own owner-gated pause	grant the adapter mint/burn rights on the token	⚠ ERC-7802 adapter no; ERC-3643 adapter untested

Project	What it owns	Bound to the token by	Light?
CMTAT-ACE	Alternative token builds (<code>ComplianceTokenCMTAT*</code>) that route protected calls through Chainlink's ACE <code>PolicyEngine</code> ; ships extractors and a <code>TransferValidationPolicy</code> that reuses CMTAT <code>IRule</code> rules	a different token deployment, not a bolt-on	✗ starts from a heavier CMTAT base than Light
CMTAT-CCIP	Foundry scripts to deploy CMTAT behind Chainlink CCIP token pools (BurnMint or LockRelease) and wire lanes, rate limiters and allowlists	CCT admin role + pool registration	✗ needs ERC-7802 / <code>CCIPModule</code>

3.1 What Light actually contains

Light is not a cut-down Standard: it is a separate build whose stated purpose is to give an issuer the smallest CMTAT that still does the job, leaving out the modules most stablecoins never use — documents, snapshots, partial freeze, debt fields, cross-chain bridges. A ✗ in the Light column below is a design decision, not an oversight.

A single base contract, `0_CMTATBaseCore`, bundles the whole feature set:

Module	Provides
<code>ERC20BaseModule</code>	ERC-20 core; updatable <code>name</code> / <code>symbol</code> ; irrevocable <code>decimals</code>
<code>ERC20MintModule</code>	<code>mint</code> , <code>batchMint</code> , <code>batchTransfer</code> (<code>MINTER_ROLE</code>)
<code>ERC20BurnModule</code>	<code>burn</code> , <code>batchBurn</code> (<code>BURNER_ROLE</code>)
<code>PauseModule</code>	<code>pause</code> / <code>unpause</code> (<code>PAUSER_ROLE</code>), <code>deactivateContract()</code> (<code>DEFAULT_ADMIN_ROLE</code>)
<code>EnforcementModule</code>	<code>setAddressFrozen</code> , <code>batchSetAddressFrozen</code> (<code>ENFORCER_ROLE</code>) — address-level only
<code>ValidationModule(Core)</code> + <code>ValidationModuleAllowance</code>	pause + freeze checks on transfer, <code>transferFrom</code> , <code>approve</code> , <code>mint</code> , <code>burn</code> ; <code>canTransfer</code> / <code>canSend</code> / <code>canReceive</code>
<code>AccessControlModule</code> , <code>VersionModule</code> , <code>TokenAttributeModule</code>	RBAC (5 roles), <code>version()</code> , name/symbol setters
defined directly in <code>CMTATBaseCore</code>	<code>forcedBurn</code> (<code>DEFAULT_ADMIN_ROLE</code> , account must be frozen first), <code>burnAndMint</code>

Two structural quirks that matter for the tables below:

- `forcedBurn` exists **only** in Light. `forcedTransfer` and `freezePartialTokens` come from `ERC20EnforcementModule`, which every variant *except* Light inherits. **No variant has both**, so an issuer must decide at deployment whether seizure means *burn* (Light) or *move* (everything else).
- `Permit` and `Standard` are mutually exclusive on ERC-2612 vs ERC-2771; the contract-size limit does not allow both. Light has neither.

4. Token standards & signature flows

Feature	Light	CMTAT	Companion	USDC	PAX	MON	FRNT	CV	USDT	EURR
---------	-------	-------	-----------	------	-----	-----	------	----	------	------

Feature	Light	CMTAT	Companion	USDC	PAX	MON	FRNT	CV	USDT	EURR
ERC-20 with standard return values	✓	✓	—	✓	✓	✓	✓	✓	✗ no return	✓
Updatable name / symbol	✓	✓	—	⚠ symbol only, at the V2.2 upgrade	✗	✗	✗	✗	✗	✗
<code>batchTransfer</code> / <code>batchMint</code> / <code>batchBurn</code>	✓	✓	—	⚠ <code>FiatTokenUtil</code> (3009 only)	✓ <code>transferFromBatch</code>	⚠ <code>BatchMint</code> helper	⚠ <code>Multicall</code>	✗	✗	✗
ERC-2612 <code>permit</code>	✗	✓ <code>Permit</code>	—	✓ (V2+)	✓	✓	✓	✗	✗	✓
ERC-3009 <code>transferWithAuthorization</code>	✗	✗ (planned, issue #346)	✗	✓	✓ + batch	✗	✗	✗	✗	✗
ERC-1271 smart-account signatures	✗	✓ <code>Permit</code> (via OZ)	—	✓ <code>SignatureChecker</code>	✓	✓	⚠ via OZ	✗	✗	✗ ECDSA only
ERC-1404 restriction codes	✗	✓ all except <code>Allowlist</code>	✓ <code>RuleEngine</code> returns them; each <code>Rules</code> rule owns a code range	✗	✗	✗	✗	✗	✗	✗
ERC-2771 meta-transactions	✗	✓ <code>Standard</code> , <code>Allowlist</code>	✓ <code>RuleEngine</code> and rule modules are ERC-2771-aware	✗	⚠ covered by 3009	✗	✗	✗	✗	✗
ERC-6357 <code>multicall</code>	✗	✓ <code>Permit</code>	—	✗	✗	✗	⚠ OZ <code>Multicall</code>	✗	✗	✗
ERC-1363 <code>transferAndCall</code>	✗	✓ <code>ERC1363</code>	—	✗	✗	⚠ ERC-677 via controller shim	✗	✗	✗	✗
ERC-7802 cross-chain mint/burn	✗	✓ all except <code>Allowlist</code>	—	✗ (CCTP)	✗	✗	✗ (LayerZero OFT)	✗	✗	✗
ERC-3643 (partial)	✗	⚠ token-side interfaces only, via <code>IERC3643Partial</code> — no on-chain identity layer	⚠ <code>Rules</code> <code>IdentityRegistryWhitelist</code> , <code>RuleIdentityRegistry</code> fill the identity slot of an ERC-3643 token, not of CMTAT	✗	✗	✗	✗	✗	✗	✗
ERC-7551 / ERC-7943	⚠ ERC-7943 errors only	✓	—	✗	✗	✗	✗	✗	✗	✗
ERC-7201 namespaced storage	✓	✓	✓ <code>RuleEngine</code> , <code>Rules</code> , <code>SnapshotEngine</code>	✗	⚠ explicit <code>BaseStorageV3</code>	✓	⚠ OZ	✗ <code>+DataLayout</code>	✗	✓ + transient storage

Reading.

- **CMTAT is the only project here implementing ERC-1404, ERC-7551, ERC-7943 or ERC-7802**, and all of them sit above Light, which implements plain ERC-20 plus batch helpers.
- **Its ERC-3643 support is partial by design.** The interface it implements is named `IERC3643Partial` and covers the token-side calls — pause, freeze, mint/burn, batch, compliance callbacks. CMTAT's own README describes it as ERC-3643 "without on-chain identity", and nothing in `cmtat/CMTAT/contracts/` references an identity registry or ONCHAINID. A deployment that must satisfy ERC-3643 in full needs an identity layer built alongside it; `Rules` ships one, but it binds to an ERC-3643 token's identity slot, not to CMTAT.
- **Every stablecoin except CoinVertible and USDT ships `permit`**; Light does not.

ERC-3009 is absent from the entire stack, Light and companion projects alike, and it is the one signature standard both Circle and Paxos ship. CMTA has it on the roadmap as a dedicated deployment version ([CMTAT issue #346](#)); the status below reflects the code pinned here, not the roadmap.

5. Supply control (issuance)

Feature	Light	CMTAT	Companion	USDC	PAX	MON	FRNT	CV	USDT	EURR
Role-gated mint	✓ <code>MINTER_ROLE</code>	✓	—	✓ minters	✓ <code>SUPPLY_CONTROLLER_ROLE</code>	✓ <code>system</code>	✓ <code>MINTER_ROLE</code>	✓ registrar only	⚠ <code>owner</code> only	✓ <code>MINTER_ROLE</code>
Role-gated burn	✓ <code>BURNER_ROLE</code>	✓ (+ <code>BURNER_FROM</code> , <code>BURNER_SELF</code>)	—	⚠ minter, own balance only	✓	✓ <code>system</code>	✓ <code>BURNER_ROLE</code>	✓ registrar only	⚠ <code>owner</code>	✓ <code>MINTER_ROLE</code> , own balance
Mint to an arbitrary address	✓	✓	—	✓ (within allowance)	✓	✓	✓	✓	✗ <code>issue()</code> credits <code>owner</code> only	⚠ authorised recipients only
Per-minter mint allowance	✗	✗ in-token	✓ <code>Rules</code> <code>RuleMintAllowance</code> — <code>setMintAllowance</code> , <code>increase/decreaseMintAllowance</code> , decremented per mint	✓ <code>minterAllowance</code>	✓	✓ + global cap	✗	✗	✗	✗
Rate-limited minting (time window)	✗	✗	⚠ <code>CMTAT-ACE</code> <code>VolumeRatePolicy</code> caps issuance per recipient per period, not per minter (see below)	✗	✓ <code>RateLimit.sol</code>	✗	✗	✗	✗	✗
Max total supply cap	✗	✗ in-token	✓ <code>Rules</code> <code>RuleMaxTotalSupply</code> (+ <code>IERC3643</code>)	✗	✗	⚠ via allowance cap	✗	✗	✗	✓ <code>_maxSupply</code>
Per-call amount cap / ticket size	✗	✗ in-token	✓ <code>CMTAT-ACE</code> <code>MaxPolicy</code> , <code>VolumePolicy</code>	✗	✗	✗	✗	✗	✗	✗
Proof-of-reserve-backed minting	✗	✗ in-token	✓ <code>Rules</code> <code>RuleChainlinkPoR</code> , or <code>CMTAT-ACE</code> <code>SecureMintPolicy</code> — both cap minting at a Chainlink PoR feed	✗	✗	✗	✗	✗	✗	⚠ structural: wrapped mode holds 1:1 reserve tokens

Feature	Light	CMTAT	Companion	USDC	PAX	MON	FRNT	CV	USDT	EURR
Minter management in a separate contract	✗ RBAC only	✗ RBAC only	✓ RuleEngine + RuleMintAllowance	✓ MasterMinter	✓ SupplyControl	✗	✗	✗	✗	⚠ registry gates recipients, not minters
Atomic burn-then-mint	✓ burnAndMint	✓	—	✗	✗	⚠ recover	✗	✗	✗	✗
Mint/burn allowed while paused	✗	✗	—	✗	✓ by design	n/a (no pause)	✗	✗	✗	✗

Reading.

- **Light gates minting on MINTER_ROLE and nothing else** — no allowance, no cap, no rate limit. Circle, Paxos and Monerium each bound how much a single compromised minter key can issue; Light does not, and neither does any other CMTAT variant on its own.
- **The companion projects close most of the gap.** RuleMintAllowance matches USDC's minterAllowance, and RuleChainlinkPoR ties issuance to an on-chain reserve attestation, which none of the seven stablecoins does.
 - Every one of those rules needs the RuleEngine, so none of it is reachable from Light. An issuer who wants USDC-grade minter controls moves to Standard or Permit, which adds roughly 11 KiB of bytecode and two external contracts to deploy and govern.
- **Paxos's rate limit has no off-the-shelf equivalent.** CMTAT-ACE's VolumeRatePolicy is the closest thing in the stack, and differs on two counts:
 - *Wrong subject.* It keys on an address supplied by the extractor, and for mint(address, uint256) the shipped MintBurnExtractor exports the **recipient**, never the caller. Attaching it to mint caps how much each address can receive per period, not how much a minter can issue; bounding the minter needs a custom extractor.
 - *Different mechanics.* VolumeRatePolicy uses a fixed tumbling period (block.timestamp / timePeriodDuration); Paxos's RateLimit.sol is a continuously refilling bucket (refillPerSecond), which never restores a full allowance at a boundary.

6. Compliance & enforcement

Feature	Light	CMTAT	Companion	USDC	PAX	MON	FRNT	CV	USDT	EURR
In-token address freeze	✓ setAddressFrozen + batch (ENFORCER_ROLE)	✓	—	✓	✓	✗ external	✗ external	✓ batch	✓	✗ moved to the registry
Global pause	✓ PauseModule	✓	—	✓	✓	✗	✓	✓	✓	✓
Permanent deactivation	✓ deactivateContract()	✓	—	✗	✗	✗	✗	✗	⚠ deprecate()	✗
Pre-trade check functions	✓ canTransfer / canSend / canReceive	✓ + detectTransferRestriction	✓ every Rule rule exposes both	✗	✗	✗	⚠ hasAccess	✓ findNotFrozen	✗	✓ isBlocked, isMintRecipient
Forced burn / seizure	✓ ForcedBurn (frozen accounts)	✗	—	✗	✓ wipeFrozenAddress	⚠ recover only	✗	✓ wipeFrozenAddress	✓ destroyBlackPunde	✓ burnFromBlockedAddress
Forced transfer to a third party	✗	✓ forcedTransfer	—	✗	⚠ wipe + re-mint	✓ recover (signature-gated)	✗	✗	✗	✗
Partial balance freeze	✗	✓ FreezePartialTokens	⚠ Rules RuleERC2990 (whitelist + frozenlist)	✗	⚠ whole address	✗	✗	⚠ whole address	⚠ whole address	✗
External / shared blacklist	✗	✓ via engine	✓ RuleEngine + Rules RuleBlacklist — one engine, several tokens	✗	✗	✓ shared validator	✓ shared registry	✗	⚠ getters for successors	✓ IAuthRegistry, policy-keyed
Allowlist / whitelist	✗	✓ Allowlist variant (ALLOWLIST_ROLE)	✓ Rules RuleWhitelist, RuleReceiverWhitelist, RuleSpenderWhitelist, RuleWhitelistWrapper	✗	✗	⚠ custom validator	⚠ pluggable registry	✗	✗	✓ mint-recipient policy
Sanctions-oracle screening	✗	✗ in-token	✓ Rules RuleSanctionsList (Chainalysis), reusable as an ACE policy via TransferValidationPolicy	✗	✗	✗	✓ via registry	✗	✗	⚠ policy-dependent; the registry holds address lists, not an oracle
Pluggable transfer-rule engine	✗ no setRuleEngine	✓ all except Allowlist	✓ RuleEngine is that engine	✗	✗	✓ IValidator	✓ IAccessRegistry	✗	✗	✓ IAuthRegistry
Per-holder balance cap	✗	✗ in-token	✓ Rules RuleMaxBalance	✗	✗	✗	✗	✗	✗	✗
Per-holder volume cap per period	✗	✗ in-token	✓ CMTAT-ACE VolumeRatePolicy (fixed tumbling period)	✗	✗	✗	✗	✗	✗	✗
Trading-hours / settlement window	✗	✗	✓ CMTAT-ACE IntervalPolicy	✗	✗	✗	✗	✗	✗	✗
Externalised pause and RBAC	✗	✗ in-token	✓ CMTAT-ACE PausePolicy, RuleBaseAccessControlPolicy (Standard variant)	✗	✗	✗	✗	✗	✗	✗ both in-token
Compliance change without redeploy	✗	⚠ swap the RuleEngine	✓ CMTAT-ACE — attach / detach / reorder policies by governance	✗	✗	⚠ swap the validator	⚠ swap the registry	✗	✗	✓ swap the policy id
Per-transfer operator approval	✗	✗ in-token	✓ Rules RuleConditionalTransferRight	✗	✗	✗	✗	⚠ redemption routing	✗	✗
Identity registry / KYC binding	✗	✗ no identity-registry slot on the token	⚠ Rules ships IdentityRegistryWhitelist and RuleIdentityHistory, but they fill an ERC-3643 token's identity slot; reaching CMTAT means screening through the RuleEngine instead	✗	✗	✗	⚠ off-chain	✗	✗	⚠ via registry policies
Standardised enforcement events	⚠ ForcedTransfer event only	✓ ERC-7551 / ERC-7943	—	✗	✗	✗	✗	✗	✗	✗

Reading. Light's compliance model is exactly USDT's: an in-token address blacklist, a global pause, and a way to destroy a blacklisted balance — a modern, role-separated reimplement of the 2017 design, plus permanent deactivation and pre-trade views that USDT lacks. That is enough for a plain fiat stablecoin, and it is the case CMTA's documentation makes.

Everything beyond that lives in `Rules`: shared blacklists, sanctions screening, whitelists, balance caps, per-transfer approval, identity binding. **All of it requires leaving Light.** Once you do, the model differs from every stablecoin here: Monerium and Wyoming each have *one* pluggable hook, while the RuleEngine runs an ordered, composable stack of them.

CMTAT has three compliance architectures, not two. Beyond in-token enforcement and the RuleEngine, `CMTAT-ACE` routes protected calls through Chainlink's ACE `PolicyEngine`, where compliance is a list of policies attached per function selector and changed by governance rather than by upgrade. It ships in two shapes:

- **Lite** — swaps the RuleEngine for the PolicyEngine on transfer validation, and keeps CMTAT's roles.
- **Standard** — policy-authoritative: ACE gates mint, burn, transfer, enforcement and admin, and the token drops `AccessControlUpgradeable` for `OwnableUpgradeable`. That is a real trade-off, giving up the granular RBAC that is otherwise one of CMTAT's advantages over USDC and USDT.

EURR externalises compliance further than any other token here. Both of its checks resolve through one external contract: `isBlocked(a)` is `!AUTH_REGISTRY.isAuthorized(transferPolicyId, a)` and `isMintRecipient(a)` is `isAuthorized(mintRecipientPolicyId, a)`. The registry is addressed by **policy id**, and both ids are settable by the token admin, so one registry serves many policies and the token chooses which governs transfers and which governs minting — closer to CMTAT-ACE's per-selector model than to Monerium's single validator. Its storage still carries `__DEPRECATED_blockedList` and `__DEPRECATED_mintRecipientList`, so V3 moved these lists out of the token.

Reading `AuthRegistry` itself settles what the policies mean:

- **A policy is typed `WHITELIST` or `BLACKLIST`, so EURR is whichever its configured policy says.** `isAuthorized` returns `policyType == WHITELIST` for a listed account and `policyType == BLACKLIST` for an unlisted one — allow-by-default under a blacklist, deny-by-default under a whitelist. The same token code is an allowlist or a blacklist depending on a policy id its admin can change.
- **Policies chain.** A policy may name a `parentPolicyId` of the same type; an account not found in the child is looked up in the parent, recursively. That composes hierarchies — a base list plus per-product overrides — which none of the other externalised designs here can express.
- **Ids 0 and 1 are reserved** as always-reject and always-allow, so an admin can disable or bypass screening by pointing the token at a constant.
- **The registry has no owner or roles of its own.** `createPolicy` is unpermissioned, and each policy is administered by the address named at creation (transferable in one step via `setPolicyAdmin`). Trust flows through the policy id the token admin selects, not through the registry contract.

One structural note, since `isAuthorized` recurses: `_validateParentPolicy` blocks self-reference and type mismatch, but `modifyParentPolicy` can point two policies at each other after the fact. A cycle would make `isAuthorized` recurse until it runs out of gas, and every EURR transfer reverts with it. Reaching that state needs the cooperating policy admins to do it, so it is a design observation rather than an exposure.

Also worth knowing: `burnFromBlockedAddress` briefly flips a transient-storage unblock flag so `_burn` can pass the contract's own transfer guard.

Light gets `forcedBurn` but not `forcedTransfer`; every other variant gets the reverse. Paxos and CoinVertible can freeze then wipe; USDT can destroy. No single CMTAT deployment can both burn from a frozen address and move its tokens elsewhere.

7. Access control & governance

Feature	Light	CMTAT	Companion	USDC	PAX	MON	FRNT	CV	USDT	EURR
Model	OZ AccessControl, 5 roles	OZ AccessControl, ~15 roles	RBAC, <code>Ownable</code> or <code>Ownable2Step</code> flavours (RuleEngine, Rules, SnapshotEngine, CMTAT-Factory); <code>Ownable</code> only for LayerZero and ACE	5 bespoke singletons	OZ <code>AccessControlDefaultAdminRules</code>	<code>Ownable2Step</code> + <code>admin/system</code>	OZ <code>AccessControl</code> , 8 roles	3 bespoke operators	single <code>owner</code>	OZ <code>AccessControlEnumerable</code> + <code>Ownable</code> , 6 roles
Roles	<code>DEFAULT_ADMIN</code> , <code>MINTER</code> , <code>BURNER</code> , <code>PAUSER</code> , <code>ENFORCER</code>	+ <code>ERC20ENFORCER</code> , <code>ALLOWLIST</code> , <code>CROSS_CHAIN</code> , <code>SNAPSHOOTER</code> , <code>DOCUMENT</code> , <code>EXTRA_INFORMATION</code> , <code>DEBT</code> , <code>FRONT_UPGRADE</code> , ...	separate managers per concern (<code>onlyRulesManager</code> , <code>onlyComplianceManager</code> , <code>onlyFunctionListManager</code>)	owner, masterMinter, pauser, blacklister, rescuer	✓ granular		✓ granular	registrar, operations, technical	—	<code>DEFAULT_ADMIN</code> , <code>MINTER</code> , <code>PAUSER</code> , <code>UNPAUSER</code> , <code>BLOCKED_ADDRESS_BURNER</code> , <code>UNWRAPPER</code>
Each role grantable to several addresses	✓	✓	✓	✗ one address each	✓		✓	✗ immutable	✗	✓
Two-step role handover	✗ OZ immediate	✗ OZ immediate	✓ <code>Ownable2Step</code> flavours of RuleEngine, every rule, SnapshotEngine and the factories; ✗ LayerZero and ACE	✗	✓ delayed admin transfer	✓	⚠ OZ	✓ <code>accept*Role</code>	✗	✗ plain <code>Ownable</code>
Timelock shipped in-repo	✗	✗	✗	✗	✓ <code>timelock-controller/</code>	✗	✗	✗	✗	✗
Roles annotated with fund-redirect risk	⚠ <code>access-control.md</code>	⚠ <code>access-control.md</code>	—	✗	✓ <code>Roles.sol</code>	⚠ <code>tokenDesign.md</code>	⚠ README	✗	✗	✗

Reading.

- **Role separation favours CMTAT.** Light already separates five independently grantable roles, against USDC's five one-address-each singletons and USDT's single `owner`.
- **Governance tooling favours Paxos.** It is the only project that ships a `TimelockController` with the token, and the only one that annotates which roles can redirect funds.
- **Ownable2Step never reaches the token.** It exists on four companion contracts (RuleEngine, Rules, SnapshotEngine, CMTAT-Factory) but not on CMTAT itself, so `DEFAULT_ADMIN_ROLE` transfers take effect immediately in every deployment, Light or not. CMTAT-LayerZero's adapters use plain `Ownable`; CMTAT-ACE's Standard build uses `OwnableUpgradeable`.

8. Upgradeability & lifecycle

Feature	Light	CMTAT	Companion	USDC	PAX	MON	FRNT	CV	USDT	EURR
Immutable (standalone)	✓ <code>CMTATStandaloneLight</code>	✓	✓ Factory deploys these too	✗	✗	✗	✗	✗	✓	✗
Transparent proxy	✓ <code>CMTATUpgradeableLight</code>	✓	✓ <code>CMTAT_LIGHT_TP_FACTORY</code> , <code>CMTAT_TP_FACTORY</code>	✓	⚠ legacy admin proxy	✗	✗	✗	✗	✗
Beacon proxy (fleet-wide upgrade)	✓	✓	✓ <code>CMTAT_LIGHT_BEACON_FACTORY</code> , <code>CMTAT_BEACON_FACTORY</code>	✗	✗	✗	✗	✗	✗	✗
UUPS proxy	✗ no Light UUPS contract	✓ <code>CMTATUpgradeableUUPS</code> (Standard branch only)	⚠ <code>CMTAT_UUPS_FACTORY</code> — no Light or Permit implementation to point at	✗	✓	✓	✓	✓	✗	✓
Deterministic addresses (CREATE2)	✓	✓	✓ CMTAT-Factory — same address across chains	✗	✗	✗	✗	✗	✗	✗
Dedicated upgrade role	✓ <code>PROXY_UPGRADE_ROLE</code>	✓	✓ factories have their own access-control flavours	⚠ proxy admin	✓	⚠ <code>owned</code>	✓ <code>UPGRADER_ROLE</code>	✓ technical	n/a	⚠ <code>DEFAULT_ADMIN_ROLE</code>
Two-step upgrade authorization	✗	✗	✗	✗	⚠ via timelock	✗	✗	✓ <code>authorizeImplementation</code> → <code>upgradeTo</code>	n/a	✗
Storage-collision strategy	✓ ERC-7201	✓ ERC-7201	✓ same	⚠ inheritance order	✓ explicit storage contracts	✓ ERC-7201	⚠ OZ	✓ <code>*DataLayout</code>	n/a	✓ ERC-7201
Rescue foreign ERC-20 / native coin	✗	✗	✗ no companion provides it	✓ <code>rescueERC20</code>	✓ <code>reclaimToken</code>	✗	✓ <code>SalvageERC20</code> + <code>SalvageGas</code>	✗	✗	✗
Version exposed on-chain	✓ <code>version()</code>	✓	✓ <code>VersionModule</code> everywhere	✗	✗	⚠ <code>CONTRACT_ID()</code>	✗	✓ <code>version()</code>	✗	✗
Irreversible shutdown	✓ <code>deactivateContract()</code>	✓	—	✗	✗	✗	✗	✗	⚠ <code>deprecate()</code>	⚠ <code>completeIntegrationOnWrapped()</code> is one-way

Reading. Light keeps almost every row here, and CMTAT-Factory is the one companion it can use. The factory gives even a Light deployment two things no stablecoin in `vendor-stablecoins/` has:

- **beacon-proxy fleet upgrades** — the natural fit for a multi-currency issuer; Monerium runs four tokens, Paxos four;
- **CREATE2 deterministic addresses** — the same token address on every chain, which every cross-chain stablecoin here solves out-of-band.

Two concrete misses, both variant-independent:

- **No UUPS Light variant** exists, and `CMTATUpgradeableUUPS` is built on the Standard branch, so a Light issuer wanting UUPS must write their own.
- **Nothing in the entire stack can recover a foreign ERC-20 sent to the token contract.** Circle, Paxos and Wyoming all ship it; CMTAT and all seven companion projects lack it.

9. Yield, cross-chain and extras

Feature	Light	CMTAT	Companion	USDC	PAX	MON	FRNT	CV	USDT	EURR
On-chain balance snapshots	✗ no <code>setSnapshotEngine</code>	✓ Snapshot variant / <code>setSnapshotEngine</code>	✓ SnapshotEngine — external or in-token, with a scheduler	✗	✗	✗	✗	✗	✗	✗
Dividend / coupon distribution	✗	✓ Debt / DebtEngine variants	✓ SnapshotEngine + IncomeVault (external)	✗	✗	✗	✗	✗	✗	✗
Native yield (rebasings balances)	✗	✗	✗	✗	✓ USDG multipliers	✗	✗	✗	✗	✗
ERC-4626 yield wrapper	✗	✗	✗	✗	✗	✗	✓ FrontierVault	✗	✗	⚠ wraps a reserve token, but not ERC-4626
Cross-chain mint/burn interface	✗	✓ ERC-7802 (<code>CROSS_CHAIN_ROLE</code>)	—	✗	✗	✗	✗	✗	✗	✗
Chainlink CCIP compatibility	✗	✓ CCIPModule (<code>getCCIPAdmin()</code>)	✓ CMTAT-CCIP — scripts for BurnMint and LockRelease token pools, lane wiring, admin-role claim	✗	⚠ audited, not in repo	✗	⚠ announced migration	✗	✗	✗
LayerZero OFT adapter	✗	✓ ERC-7802 variants	✓ CMTAT-LayerZero <code>LayerZeroAdapterERC7802</code> (recommended) / <code>LayerZeroAdapter</code> (ERC-3643)	✗	✗	✗	✓ <code>FrontierOFTAdapter*</code>	✗	✗	✗
Bridge mechanism	—	—	mint/burn via LayerZero; either mint/burn or lock/release via CCIP pools	CCIP mint/burn	—	—	both: lock/unlock on the hub, mint/burn on spokes	—	—	—
Bridge-level pause	—	—	✓ owner-gated <code>pause()</code> on each LayerZero adapter	⚠ token-level only	—	—	⚠ token-level only	—	—	—
Bridge-level rate limit / allowlist	—	—	✓ CMTAT-CCIP — per-lane CCIP pool rate limiters and allowlists	✗	✗	✗	✗	✗	✗	✗
Non-EVM implementation	✗	✓ CMTAT-Tezos-FA2 (SmartPy) and private-CMTAT-aztec (Noir) vendored here; CMTAT-Solana is a specification, not a program; Canton external	—	✗	✗	✗	✓ Solana program in-repo	✗	⚠ separate codebases	✗
Confidential balances	✗	✓ two routes: CMTAT-Confidential (Zama FHE, EVM) and private-CMTAT-aztec (Aztec L2, Noir, prototype)	—	✗	✗	✗	✗	✗	✗	✗
On-chain legal documents	✗	✓ ERC-1643 <code>DocumentERC1643Module</code>	—	✗	✗	✗	⚠ <code>contractUri</code>	✗	✗	✗
On-chain token metadata (ISIN, terms)	✗	✓ <code>ExtraInformationModule</code>	—	✗	✗	✗	✗	✗	✗	✗
On-chain holder registry	✗	✓ <code>HolderList</code> variant	⚠ Rules whitelists approximate it	✗	⚠ payout groups	✗	✗	✗	✗	✗
Transfer fee	✗	✗	✗	✗	✗	✗	✗	✗	✓ (set to 0)	✗
Redemption modelled on-chain	⚠ burn	⚠ burn	⚠ <code>RuleConditionalTransferLight</code> can gate it	⚠ burn	⚠ burn	⚠ signature-gated burn	⚠ burn	✓ redemption addresses	⚠ <code>redeem</code>	✓ <code>unwrap</code> returns the reserve token

Reading.

- **Light is stripped of all of this, and for a plain fiat stablecoin that is the right call.** None of the seven stablecoins has documents, snapshots or holder lists either.
- **What Light also gives up, less obviously, is cross-chain.** ERC-7802 and the CCIP module are absent from it, so a Light token has no standard bridge entry points. Five of the seven stablecoins here are multi-chain (CoinVertible and EURR are the exceptions), which makes this the most likely reason to leave Light for reasons unrelated to compliance.

No part of the CMTAT stack pays holders a yield: neither the rebasing model (Paxos USDG) nor the ERC-4626 wrapper (Wyoming wFRNT) has a counterpart. SnapshotEngine + IncomeVault addresses periodic *distribution*, which is a different mechanism from continuously accruing balances.

10. Gap analysis

10.1 Stablecoin features the CMTA stack covers outside the token

These are all available to a CMTAT issuer, but none of them live in the token contract: each needs a companion project deployed and wired alongside it. The last column is the one that matters in practice, because CMTA's own guidance points stablecoin issuers at Light.

Feature	Who has it	CMTAT answer	From Light?
---------	------------	--------------	-------------

Feature	Who has it	CMTAT answer	From Light?
Per-minter mint allowance	USDC, Paxos, Monerium	✓ Rules <code>RuleMintAllowance</code>	✗ needs RuleEngine
Shared blacklist across several tokens	Monerium, Wyoming	✓ RuleEngine + Rules <code>RuleBlacklist</code> (heed the stateful-rule warning)	✗ needs RuleEngine
Sanctions-oracle screening	Wyoming	✓ Rules <code>RuleSanctionsList</code>	✗ needs RuleEngine
Minter management in a separate contract	USDC (<code>MasterMinter</code>), Paxos (<code>SupplyControl</code>)	✓ RuleEngine + <code>RuleMintAllowance</code> is the architectural equivalent	✗ needs RuleEngine
Per-transfer operator approval	CoinVertible (registrar validation)	✓ Rules <code>RuleConditionalTransferLight</code>	✗ needs RuleEngine
Max supply / per-holder balance caps	none	✓ Rules <code>RuleMaxTotalSupply</code> , <code>RuleMaxBalance</code>	✗ needs RuleEngine
Proof-of-reserve-gated minting	none	✓ Rules <code>RuleChainlinkPoR</code> , or CMTAT-ACE <code>SecureMintPolicy</code>	✗ needs RuleEngine or an ACE build
Deterministic cross-chain addresses	none — all solve it out-of-band	✓ CMTAT-Factory (CREATE2)	✓ yes
Fleet-wide upgrade of many tokens	none — all upgrade per token	✓ CMTAT-Factory beacon factories	✓ yes
LayerZero OFT bridging	Wyoming	✓ CMTAT-LayerZero adapters	⚠ recommended adapter needs ERC-7802; the fallback is untested on Light
Per-recipient volume cap per period	none	✓ CMTAT-ACE <code>VolumeRatePolicy</code>	✗ separate token build
Compliance changed by configuration, not upgrade	Monerium, Wyoming (one hook each)	✓ CMTAT-ACE policy attach/detach/reorder	✗ separate token build

CMTAT-Factory is the only companion project a Light deployment can definitely use:

- RuleEngine and every rule need `setRuleEngine`, which Light does not have.
- SnapshotEngine needs either `setSnapshotEngine`, also absent, or its in-token variants, which start from a heavier CMTAT base.
- CMTAT-LayerZero's ERC-3643 adapter may work with Light, but nothing in that repository tests it.

10.2 Stablecoin features the CMTA ecosystem does not provide

The genuine gaps: absent from every CMTAT variant **and** from all seven companion projects, so an issuer who needs one has to build it.

Feature	Who has it	Notes
ERC-3009 <code>transferWithAuthorization</code>	USDC, Paxos	The main payments-integration gap. Not in any CMTAT variant, nor in any of the seven companion projects. Planned: CMTA tracks it as a dedicated deployment version in issue #346 .
Per-minter rate limiting	Paxos <code>RateLimit.sol</code>	<code>RuleMintAllowance</code> is a quota with no time dimension. CMTAT-ACE's <code>VolumeRatePolicy</code> is time-based but keys on the mint recipient, not the minter, so bounding a minter needs a custom ACE extractor.
Rescue / salvage of foreign tokens	USDC, Paxos, Wyoming	Absent from CMTAT <i>and</i> all seven companion projects.
Native yield / rebasing balances	Paxos USDG	No CMTAT equivalent; snapshots address distribution, not accrual.
ERC-4626 yield wrapper	Wyoming wFRNT	Would have to be built on top of CMTAT.
Signature-gated burn / wallet recovery	Monerium	<code>forcedTransfer</code> is unilateral; Monerium requires the holder's signature.
Transfer fee	USDT	Deliberately absent; noted for completeness.
<code>forcedBurn</code> and <code>forcedTransfer</code> together	Paxos, CoinVertible (freeze → wipe)	Both exist in the stack, but never in one deployment: <code>forcedBurn</code> is Light-only, <code>forcedTransfer</code> is every-variant-but-Light. The gap is the combination, not either function.
Timelock shipped with the token	Paxos	No CMTA project ships one. OpenZeppelin's <code>TimelockController</code> drops in, so this is the cheapest of the gaps to close.

10.3 CMTAT features no stablecoin in `vendor-stablecoins/` has

- **ERC-1404 restriction codes** and pre-trade `canTransfer` / `detectTransferRestriction` views — every stablecoin here simply reverts.
- **Composable rule stacking** — Monerium and Wyoming each have *one* pluggable hook; RuleEngine runs an ordered list of them.
- **Proof-of-reserve-gated minting** (`RuleChainlinkPoR`) — no stablecoin here ties issuance to an on-chain reserve attestation.
- **Per-holder balance caps** and **max total supply caps** as policy rather than code.

- **Permanent deactivation** (`deactivateContract()`, ERC-8343 draft) — only USDT's `deprecate()` is comparable, and it forwards rather than terminates.
- **On-chain snapshots** for dividend distribution (SnapshotEngine).
- **On-chain legal documents** (ERC-1643) and token metadata (ISIN, terms).
- **Standardised enforcement events** (ERC-7551 / ERC-7943).
- **Confidential balances**, by two different routes: `CMTAT-Confidential` keeps the token on the EVM and encrypts balances with Zama FHE; `private-CMTAT-aztec` reimplements it in Noir on the Aztec privacy L2, where transactions stay private to holders while the issuer keeps an audit view. The Aztec build is an unaudited prototype and its README warns that Aztec itself is still changing rapidly.
- **Beacon + CREATE2 factory deployment** (CMTAT-Factory).
- **A bridge that can be paused independently of the token** — CMTAT-LayerZero's adapters each carry their own owner-gated `pause()`. Wyoming's OFT adapters have no such switch; halting a bridge there means pausing the token.
- **The standard is genuinely blockchain-agnostic, and the non-EVM work exists.** Every stablecoin here is Solidity-only, or ports by writing a new codebase from scratch. CMTAT is defined as a feature set and reimplemented per chain:
 - `CMTAT-Tezos-FA2` — a SmartPy implementation on Tezos' FA2 standard, from the Tezos Foundation, audited by Inference. FA2 holds several tokens per contract, so it renames CMTAT's `owner` to *token admin* and extends entry points to take lists for batching. Pinned at 2023, well before CMTAT v3, so treat it as a separate lineage rather than a port of the code compared here.
 - `private-CMTAT-aztec` — Noir, on the Aztec privacy L2.
 - `CMTAT-Solana` — a v1.0.0 specification mapping the feature set onto SPL Token-2022 extensions (permanent delegate, transfer hook, default account state). Guidance, not a program: Wyoming ships the only running Solana code here, and that is a bridge endpoint rather than the token.
- **Bridge-agnostic cross-chain entry points** — ERC-7802 is a standard interface any bridge can drive; every stablecoin here is wired to one specific bridge (CCTP for Circle, LayerZero for Wyoming). CMTAT has working tooling for both LayerZero and CCIP.
- **Compliance as runtime configuration** (CMTAT-ACE) — policies are attached, detached and reordered by governance per function selector. Monerium and Wyoming can swap their single hook; no stablecoin here can reorder a policy chain without an upgrade.
- **Trading-hours windows** (`IntervalPolicy`) and **per-holder rolling volume caps** (`VolumeRatePolicy`) — neither has any counterpart in `vendor-stablecoins/`.
- **A token that can be recomposed rather than forked.** 25 modules and a layered base chain mean an issuer can assemble a variant that matches their requirements, and an integrator can add features CMTAT never shipped — as `SnapshotEngine` and `CMTAT-ACE` both do. The stablecoins here are fixed inheritance chains; Paxos comes closest with its diamond facets, but those are Paxos's own facets, not a kit anyone else builds with.
- **Mutable** `name` / `symbol` post-deployment.

10.4 Practical conclusion for a stablecoin issuer

CMTAT offers **Light** to issuers who want a minimal contract rather than the full feature set, and recommends it for stablecoins. The question is therefore not whether Light is missing things — it is missing them on purpose — but whether the things it leaves out are ones a given stablecoin needs.

Where Light is sufficient. For a single-chain, non-yield-bearing fiat token whose compliance need is "freeze an address and, if ordered, destroy its balance", Light matches USDT's model and adds:

- five independently grantable roles instead of one `owner`;
- batch operations and pre-trade `canTransfer` views;
- permanent deactivation and mutable `name` / `symbol`;
- through CMTAT-Factory, beacon upgrades and CREATE2 addresses that no stablecoin in `vendor-stablecoins/` has.

At 11.3 KiB it is also the smallest deployment in the set.

Where Light falls short. Four things the stablecoins here provide and Light does not:

What Light lacks	Who has it	Cost of fixing it
Any bound on minter authority	USDC, Paxos, Monerium	move to Standard/Permit + RuleEngine + <code>RuleMintAllowance</code>
<code>permit</code> (ERC-2612)	USDC, Paxos, Monerium, Wyoming	move to Permit — which forfeits ERC-2771
Cross-chain entry points	Circle (CCTP), Wyoming (OFT), Paxos	move to a variant that implements ERC-7802 / CCIP, then add a <code>CMTAT-LayerZero</code> adapter
Forced <i>transfer</i> (it has forced <i>burn</i>)	Paxos, CoinVertible	move to any non-Light variant — and lose <code>forcedBurn</code>

The competitive configuration is Standard or Permit + RuleEngine + Rules, which matches USDC and Monerium feature for feature and adds composable rule stacking neither has, at a reported 22–23 KiB plus two external contracts, roughly double Light's footprint.

Four gaps survive every configuration and must be built by the issuer: **ERC-3009**, **per-minter rate limiting**, **foreign-token rescue**, and **yield accrual**. Only ERC-3009 has a published plan, as a dedicated deployment version in [CMTAT issue #346](#).

See also

- [CHANGELOG.md](#) — what changed in each version of this analysis.
- [vendor-stablecoins/README.md](#) — per-directory guide to each stablecoin codebase and its main files.
- [vendor-stablecoins/SUMMARY.md](#) — stablecoin-to-stablecoin comparison.
- [stablecoin-doc-issue.md](#) — 14 documented errors in CMTAT's own stablecoin comparison, with suggested fixes.
- [cmtat/CMTAT/doc/technical/stablecoin.md](#) — CMTAT's Light-variant deployment guide.
- [cmtat/Rules/README.md](#) — the rule catalogue and restriction codes.
- [cmtat/RuleEngine/README.md](#) — engine architecture and token-binding model.
- [cmtat/CMTAT-LayerZero/README.md](#) — OFT adapter selection and deployment guide.

AI assistance

Parts of this project were written with the help of AI coding assistant Claude Code (Anthropic).